

Projeto e Análise de Algoritmos I

Aula 04 - Análise de Algoritmos e Notação Assintótica

Lucas Nunes Alegre

Universidade Federal do Rio Grande do Sul
Instituto de Informática
Departamento de Informática Teórica

Medindo o Custo de Algoritmos

Como Medir o Custo de um Algoritmo?

Quando analisamos algoritmos, precisamos de uma métrica de **custo**.

Mas **o que** exatamente devemos medir?

Possíveis Medidas:

- Tempo de execução (*ms, segundos, minutos, dias, anos*)
- Espaço utilizado na memória (*bits, KB, MB, GB, TB*)
- Consumo de energia (*kWh*)
- Custos financeiros (\$, R\$, £, ¥)
- Outros recursos (rede, *hardware* especializado, etc.)

P: Qual dessas medidas é a mais relevante para analisar algoritmos?

Modelo de Computação

Modelo RAM (Random Access Machine):

- Vamos assumir um computador genérico, que executa operações em sequência, sem paralelismo.
- E se o processador tem uma instrução para ordenação? Ordenação $O(1)$? **Não realístico.**
- Exponenciação é operação de tempo constante?
 - Pode ser implementado com *shift-right/shift-left*.
 - Vamos assumir que 2^k tem tempo constante para k pequeno.
- Não vamos considerar hierarquia de memória.

Modelo RAM (Random Access Machine)

Obs. Em geral é desnecessário definir o custo de cada instrução, assumimos um **custo constante** para qualquer instrução.

Def. Toda operação simples leva tempo constante:

- Adição, subtração, multiplicação, divisão, ...
- Movimentos de dados (acesso e escrita na memória).
- Controle condicional.

Obs. Operações complexas não levam tempo constante:

- *Loops*.
- Chamadas a sub-rotinas como ordenação.

Análise de Algoritmos

P: Como analisar o tempo de execução de um algoritmo?

Obs. O tempo requerido por um algoritmo depende da **entrada**:

- Ordenar 100.000 números leva mais tempo que ordenar 3 números.

Precisamos levar o **tamanho da entrada** em consideração.

Análise de Algoritmos: Tamanho da Entrada

Tamanho da Entrada:

- Depende do problema a ser estudado.
- Geralmente o número de itens na entrada (números).
- Multiplicar dois números inteiros: número de bits.
- Pode ser descrito por mais de um número: grafos (V e E).

P: Qual o tamanho da entrada de um Cubo Mágico?

Análise de Algoritmos: Tempo de Execução

Def. (Tempo de Execução): Para uma entrada específica, é o número de instruções primitivas (*steps*) executadas.

Instruções independentes de máquina:

- Cada linha i custa tempo constante c_i .
- Uma linha que custa c_i e é executada n vezes vai contribuir $c_i \cdot n$.

$T(n)$ = número de instruções executadas para entrada de tamanho n .

Laços (Loops)

O teste do laço é executado uma vez a mais que o conteúdo do bloco:

ZERA-VETOR(A, n)

- 1: **for** $j \leftarrow 1$ **to** n **do** // Executa $n + 1$ vezes
 - 2: $A[j] \leftarrow 0$ // Executa n vezes
-

EXEMPLO-WHILE(j)

- 1: **while** $j > 0$ **do** // Executa $X + 1$ vezes
 - 2: $j \leftarrow \dots$ // Executa X vezes
-

Análise do Insertion Sort

Ordenação por Inserção (Insertion Sort)

INSERTIONSORT(A)

```
1:  for  $j \leftarrow 2$  to length( $A$ ) do
2:     $key \leftarrow A[j]$ 
3:    // Insere  $A[j]$  em  $A[1 \dots j - 1]$ 
4:     $i \leftarrow j - 1$ 
5:    while  $i > 0$  and  $A[i] > key$  do
6:       $A[i + 1] \leftarrow A[i]$ 
7:       $i \leftarrow i - 1$ 
8:     $A[i + 1] \leftarrow key$ 
```

Linha	Custo	Repetições
1.	c_1	n
2.	c_2	$n - 1$
3.	0	$n - 1$
4.	c_4	$n - 1$
5.	c_5	$\sum_{j=2}^n t_j$
6.	c_6	$\sum_{j=2}^n (t_j - 1)$
7.	c_7	$\sum_{j=2}^n (t_j - 1)$
8.	c_8	$n - 1$

t_j : nº de repetições do laço da linha 5 para aquele valor de j .

Ordenação por Inserção (Insertion Sort)

Somando todos os custos:

$$T(n) = c_1 n + c_2(n - 1) + 0(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1)$$

Problema: Como nos livramos de t_j ? O valor de t_j para cada j depende da sequência de entrada, que não temos como saber *a priori*.

Solução: Podemos analisar o **melhor** e o **pior** caso! Isto é, podemos inferir limites inferiores e limites superiores para t_j .

Ordenação por Inserção (Insertion Sort)

- Melhor Caso: $t_j = 1$
- $A[i] \leq key$
- Sequência já ordenada!

$$T(n) = c_1 n + c_2(n - 1) + 0(n - 1) + c_4(n - 1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1) + c_7 \sum_{j=2}^n (t_j - 1) + c_8(n - 1)$$

Ordenação por Inserção (Insertion Sort)

- Melhor Caso: $t_j = 1$
- Sequência já ordenada! $A[i] \leq key$

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + 0(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n 1 + \\ &\quad c_6 \sum_{j=2}^n (1-1) + c_7 \sum_{j=2}^n (1-1) + c_8(n-1) \\ &= c_1 n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \\ &= a \cdot n + b \end{aligned}$$

Lembrando que $\sum_{j=2}^n 1 = n - 1$.

Melhor Caso: Função linear em n .

Ordenação por Inserção (Insertion Sort)

- Pior Caso: $t_j = j$
- $A[i] > key$ sempre
- Sequência ordenada em ordem reversa!

$$\sum_{j=2}^n j = \frac{n(n+1)}{2} - 1 \quad \text{e} \quad \sum_{j=2}^n (j-1) = \frac{n(n-1)}{2}$$

$$\begin{aligned} T(n) &= c_1 n + c_2(n-1) + c_4(n-1) + c_5 \left(\frac{n(n+1)}{2} - 1 \right) + \\ &= c_6 \left(\frac{n(n-1)}{2} \right) + c_7 \left(\frac{n(n-1)}{2} \right) + c_8(n-1) \\ &= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2} \right) n^2 + \left(c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8 \right) n - (c_5 + c_6 + c_7) \\ &= an^2 + bn + c \end{aligned}$$

Ordenação por Inserção (Insertion Sort)

Pior Caso: $t_j = j$

Pior Caso: Função quadrática em n .

- a , b e c dependem do custo das instruções (c_i).
- $T(n) = n^2$?
- Não é igual, mas cresce como n^2 .
- $T(n) \sim n^2$ (ordem de crescimento).

Análise de Algoritmos

Análise de Algoritmos

P: Por que o pior caso?

Obs. Nós em geral nos preocupamos em encontrar o tempo de execução do **pior caso**:

- O maior tempo de execução para **qualquer entrada** de tamanho n .
- O pior caso dá uma garantia para **qualquer entrada**.
- Para alguns algoritmos o pior caso ocorre com frequência:
 - Exemplo: busca de um item não presente em um vetor.

P: Alternativas ao pior caso?

Análise de Algoritmos

P: Qual é o caso médio do algoritmo de Ordenação por Inserção?

Obs. Para ordenar um vetor ordenado randomicamente com elementos distintos:

- Em média $A[j]$ é menor que metade dos elementos em $A[1 \dots j - 1]$.
- Assim, o algoritmo deve comparar o elemento com metade dos elementos até encontrar a posição de *key*.

Obs. Então $t_j = \frac{j}{2}$, isso é aproximadamente metade do pior caso, mas ainda é uma **função quadrática**.

P: Quais instâncias aparecem no mundo real?

Busca em Vetor Não Ordenado

No nosso caso, abstraímos com unidades constantes de computação.

BUSCA-LINEAR(x , $A[1..n]$)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = x$  then
3:          return  $i$ 
4:  return NÃO-ENCONTRADO
```

Passos unitários contados:

- leitura de $A[i]$
- comparação $A[i] = x$
- incremento de i
- return

Obs. Nessa disciplina, também iremos considerar unidades constantes de armazenamento para analisar a complexidade de espaço.

Busca em Vetor Não Ordenado

BUSCA-LINEAR(x , $A[1..n]$)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = x$  then
3:          return  $i$ 
4:  return NÃO-ENCONTRADO
```

Exemplo: Procurar $x = 23$ em $A[1..10]$

Índice	1	2	3	4	5	6	7	8	9	10
Valor	42	7	85	13	66	90	23	5	77	31

Busca em Vetor Não Ordenado

BUSCA-LINEAR(x , $A[1..n]$)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = x$  then
3:          return  $i$ 
4:  return NÃO-ENCONTRADO
```

Exemplo: Procurar $x = 42$ em $A[1..10]$

Índice	1	2	3	4	5	6	7	8	9	10
Valor	42	7	85	13	66	90	23	5	77	31

Busca em Vetor Não Ordenado

BUSCA-LINEAR(x , $A[1..n]$)

```
1:  for  $i \leftarrow 1$  to  $n$  do  
2:      if  $A[i] = x$  then  
3:          return  $i$   
4:  return NÃO-ENCONTRADO
```

Exemplo: Procurar $x = 15$ em $A[1..10]$

Índice	1	2	3	4	5	6	7	8	9	10
Valor	42	7	85	13	66	90	23	5	77	31

Qual Caso Devemos Considerar?

- **Melhor Caso:** quando encontramos o elemento logo na primeira comparação.
- **Pior Caso:** quando precisamos examinar todos os elementos do vetor.
- **Caso Médio:** quando o elemento pode aparecer em qualquer posição com igual probabilidade.

P: Qual desses casos é o mais relevante para analisar algoritmos em teoria da computação?

Análise de Pior Caso

- Procuramos um limite superior para o **tempo de execução máximo** de um algoritmo em entradas de tamanho N .
- Pode parecer drástico, mas na prática a análise de **pior caso** costuma refletir bem a eficiência real.
- Alternativa: análise de caso médio (entradas “aleatórias”), **porém**:
 - Difícil definir uma distribuição de entradas realista.
 - Resultados dependem mais do modelo de aleatoriedade do que do algoritmo.
- Análise de **pior caso** é a **abordagem padrão** na teoria de algoritmos.

Benchmark inicial: comparação com **busca força bruta** sobre todo o espaço de soluções.

Busca em Vetor Não Ordenado

P: Qual o pior caso?

BUSCA-LINEAR(x , $A[1..n]$)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = x$  then
3:          return  $i$ 
4:  return NÃO-ENCONTRADO
```

$$T(n) = 3n + 1$$

Bubble Sort

P: Qual o pior caso?

TROCA(A, x, y)

```
1:   $tmp \leftarrow A[x]$   
2:   $A[x] \leftarrow A[y]$   
3:   $A[y] \leftarrow tmp$ 
```

$$T(n) = 2 + 2 + 1 = 5$$

BUBBLESORT(A)

```
1:   $n \leftarrow \text{length}(A)$   
2:  for  $i \leftarrow 1$  to  $n - 1$  do  
3:    for  $j \leftarrow 1$  to  $n - i$  do  
4:      if  $A[j] > A[j + 1]$  then  
        TROCA( $A, j, j + 1$ )
```

$$T(n) = (n - 1)(n - 1)(1 + 5) = 6n^2 - 12n + 6$$

Ordem de Crescimento

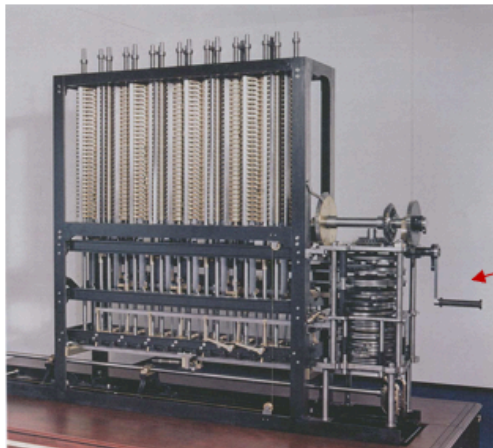
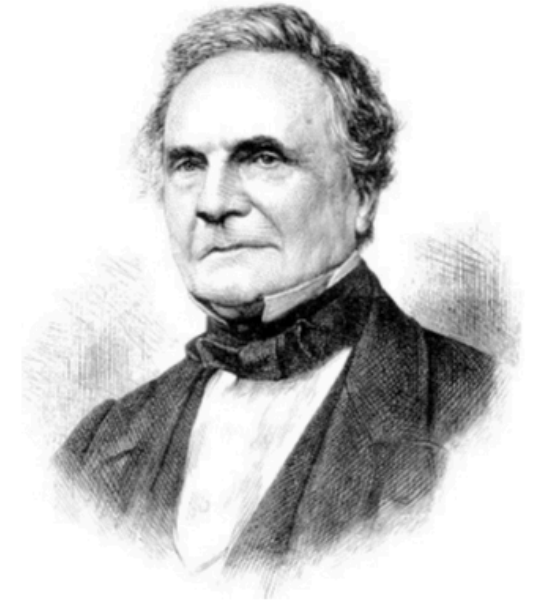
Obs. Focar a análise nas características importantes.

- Considerar apenas os termos de **maior ordem** da fórmula.
- Remover os termos de menor ordem.
- Ignorar os coeficientes (constantes).

Um Pensamento Moderno

"Assim que um Motor Analítico existir, ele necessariamente guiará o futuro curso da ciência. Sempre que qualquer resultado for buscado com sua ajuda, surgirá a questão — Por qual curso de cálculo esses resultados podem ser obtidos pela máquina no menor tempo?"

— Charles Babbage (1864)



Quantas vezes temos
que girar a alavanca?

Tempos de Execução

Eficiência: Diferentes algoritmos para resolver o mesmo problema podem diferir dramaticamente em sua eficiência.

Problema de Ordenação: Uma sequência de $n = 10.000.000$ números.

Algoritmo A: $T(n) = 2n^2$.

Algoritmo B: $T(n) = 50n \log n$.

Constantes $c_1 = 2$ e $c_2 = 50$ não dependem de n .

P: Qual será a diferença de eficiência dos dois algoritmos?

Tempos de Execução

Computador 1: Executa 10 bilhões de instruções por segundo.

$$\text{A-1: } \frac{2 \cdot (10^7)^2 \text{ instruções}}{10^{10} \text{ instruções/segundo}} = 20.000 \text{ segundos (5,5 horas).}$$

$$\text{B-1: } \frac{50 \cdot (10^7) \cdot \log(10^7) \text{ instruções}}{10^{10} \text{ instruções/segundo}} = 1,16 \text{ segundos.}$$

Computador 2: Executa 10 milhões de instruções por segundo.

$$\text{A-2: } \frac{2 \cdot (10^7)^2 \text{ instruções}}{10^7 \text{ instruções/segundo}} = 20.000.000 \text{ segundos (231 dias).}$$

$$\text{B-2: } \frac{50 \cdot (10^7) \cdot \log(10^7) \text{ instruções}}{10^7 \text{ instruções/segundo}} = 1.163 \text{ segundos } (\approx 19 \text{ minutos}).$$

Tempos de Execução

	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	4 sec
$n = 30$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	18 min	10^{25} years
$n = 50$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	11 min	36 years	very long
$n = 100$	< 1 sec	< 1 sec	< 1 sec	1 sec	12,892 years	10^{17} years	very long
$n = 1,000$	< 1 sec	< 1 sec	1 sec	18 min	very long	very long	very long
$n = 10,000$	< 1 sec	< 1 sec	2 min	12 days	very long	very long	very long
$n = 100,000$	< 1 sec	2 sec	3 hours	32 years	very long	very long	very long
$n = 1,000,000$	1 sec	20 sec	12 days	31,710 years	very long	very long	very long

Tempo de execução para diferentes algoritmos em entradas de tamanho crescente em um computador executando um milhão de instruções por segundo. Em casos com tempo $\geq 10^{25}$ anos substituído por *very long*. Para referência, a idade estimada do universo é de aproximadamente 4×10^{17} segundos.

Eficiência: Força Bruta

Espaço de Busca:

- Os problemas que vamos estudar possuem **natureza discreta**.
- Eles envolvem uma **busca implícita** sobre um grande conjunto de **possibilidades combinatórias**.
- Com o objetivo de encontrar uma **solução** que satisfaz um conjunto de condições.
- Para muitos problemas existirá a opção de realizar uma **busca por força bruta** checando todas as soluções.

P: Exemplo?

Obs. Em geral, nós consideramos o algoritmo mais eficiente se seu tempo de execução no **pior caso** tem *ordem de crescimento menor* (tese de Cobham-Edmonds).

P: Uma definição para um algoritmo eficiente?

Eficiência: Polinomial

Def. Nós dizemos que um algoritmo é **eficiente** se tem tempo de execução **polinomial**.

P: E se for n^{100} vs. 1.001^n ?

Justificativa:

- Funciona na prática.
- Normalmente, algoritmos têm baixas constantes e baixos expoentes.
- Desenvolver um algoritmo polinomial em geral envolve entender alguma estrutura fundamental do espaço de busca do problema.

Exexceptions:

- Simplex vs. Método do Elipsoide.
- Busca Heurística.

Análise Assintótica

Intuição

Ideia: Suprimir fatores constantes e termos de baixa ordem.

- Independente de máquina.
- Foco em grandes entradas.
- Descreve o crescimento de funções.
- Descreve o comportamento de funções **no limite**.

Exemplo: $8n^3 + 50n + 86$ cresce como n^3 .

Terminologia: O algoritmo tem tempo de execução $O(n^3)$, onde n é o tamanho da entrada.

Intuição

Problema: O vetor A contém o número t ?

$\text{BUSCA}(A, n, t)$

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = t$  then
3:          return true
4:  return false
```

P: Qual é o tempo de execução?

P: Quanto espaço é usado?

Intuição

Problema: O vetor A contém o número t ?

BUSCA(A, n, t)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = t$  then
3:          return true
4:  return false
```

P: Qual é o tempo de execução? $O(n)$

Intuição

Problema: O vetor A ou o vetor B contém o número t ?

BUSCA-DUPLA(A, B, n, t)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = t$  then
3:          return true
4:  for  $i \leftarrow 1$  to  $n$  do
5:      if  $B[i] = t$  then
6:          return true
7:  return false
```

P: Qual é o tempo de execução?

Intuição

Problema: O vetor A ou o vetor B contém o número t ?

BUSCA-DUPLA(A, B, n, t)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = t$  then
3:          return true
4:  for  $i \leftarrow 1$  to  $n$  do
5:      if  $B[i] = t$  then
6:          return true
7:  return false
```

P: Qual é o tempo de execução? $O(n)$

Intuição

Problema: O vetor A e o vetor B contêm um número em comum?

BUSCA-COMUM(A, B, n)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      for  $j \leftarrow 1$  to  $n$  do
3:          if  $A[i] = B[j]$  then
4:              return true
5:  return false
```

P: Qual é o tempo de execução?

Intuição

Problema: O vetor A e o vetor B contêm um número em comum?

BUSCA-COMUM(A, B, n)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      for  $j \leftarrow 1$  to  $n$  do
3:          if  $A[i] = B[j]$  then
4:              return true
5:  return false
```

P: Qual é o tempo de execução? $O(n^2)$

Intuição

Problema: O vetor A tem um elemento duplicado?

VERIFICA-DUPLICATAS(A, n)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      for  $j \leftarrow i + 1$  to  $n$  do
3:          if  $A[i] = A[j]$  then
4:              return true
5:  return false
```

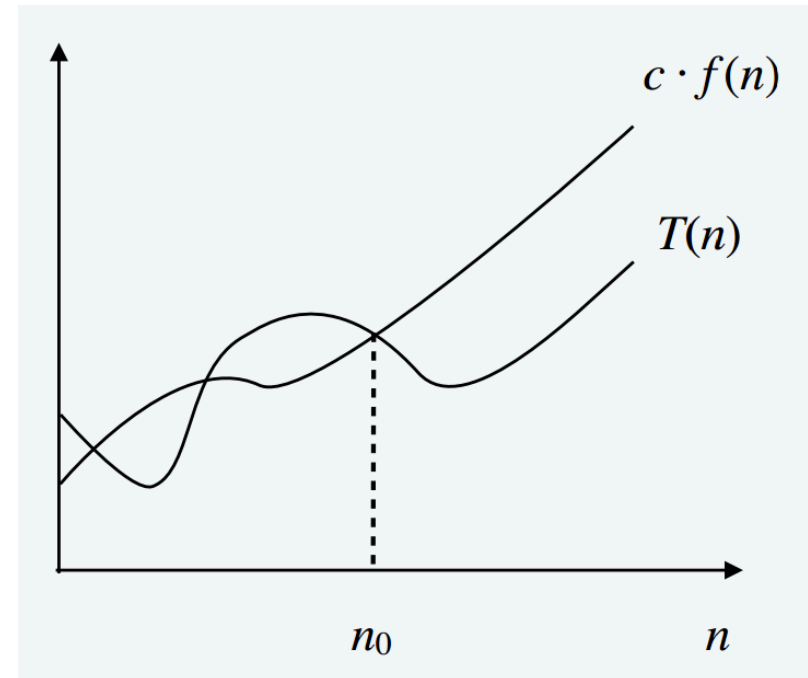
P: Qual é o tempo de execução? $O(n^2)$

Notação Big O - Limitante Superior

Def. $T(n)$ é $O(f(n))$ se e somente se **existe** uma constante $c > 0$ e um $n_0 \geq 0$ tal que $T(n) \leq c \cdot f(n)$ **para todo** $n \geq n_0$.

Exemplo: $T(n) = 32n^2 + 17n + 1$

- $T(n)$ é $O(n^2)$.
- $T(n)$ é $O(n^3)$.
- $T(n)$ **não** é $O(n)$.
- $T(n)$ **não** é $O(n \log(n))$.
- **Uso:** Ordenação por Inserção é $O(n^2)$.



Obs. c e n_0 não podem depender de n .

Notação Big O - Limitante Superior

Def. $T(n)$ é $O(f(n))$ se e somente se **existe** uma constante $c > 0$ e um $n_0 \geq 0$ tal que $T(n) \leq c \cdot f(n)$ para todo $n \geq n_0$.

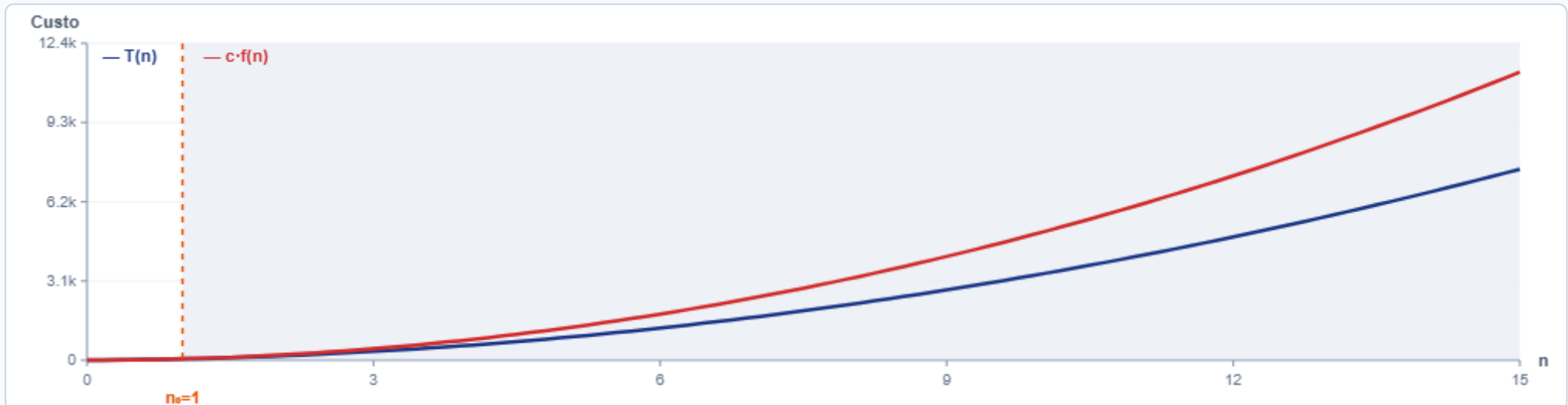
Demonstração Interativa Big-O

Função $T(n)$ e candidato $f(n)$:

$T(n) = 32n^2 + 17n + 1$ | $f(n) = n^2$ [$O(n^2)$]

Constante $c = 50$

Limiar $n_0 = 1$



✓ $T(n) \leq 50 \cdot f(n)$ para todo $n \geq 1$

Busca em Vetor Não Ordenado

BUSCA-LINEAR($x, A[1..n]$)

```
1:  for  $i \leftarrow 1$  to  $n$  do
2:      if  $A[i] = x$  then
3:          return  $i$ 
4:  return NÃO-ENCONTRADO
```

$$T(n) = 3n + 1$$

$T(n)$ é $O(n)$: $c = 4$ e $n_0 = 1$, $c = 3,5$ e $n_0 = 2$, ...

$T(n)$ é $O(n^2)$, $O(n^3)$, ...

<https://www.geogebra.org/classic/czf4djzt>

Bubble Sort

BUBBLESORT(A)

```
1:   $n \leftarrow \text{length}(A)$ 
2:  for  $i \leftarrow 1$  to  $n - 1$  do
3:    for  $j \leftarrow 1$  to  $n - i$  do
4:      if  $A[j] > A[j + 1]$  then
        TROCA( $A, j, j + 1$ )
```

$$T(n) = (n - 1)(n - 1)(1 + 5) = 6n^2 - 12n + 6$$

$T(n)$ é $O(n^2)$: $c = 6$ e $n_0 = 0, \dots$

$T(n)$ é $O(n^3)$, $O(n^4)$, $\dots$

$T(n)$ não é $O(n)$.

<https://www.geogebra.org/classic/h2f2scrd>

Notação Big O : Polinômios

Def. $T(n)$ é $O(f(n))$ se e somente se existe uma constante $c > 0$ e um n_0 tal que $T(n) \leq c \cdot f(n)$ para todo $n \geq n_0$.

Afirmção. Se $T(n) = a_k n^k + \dots + a_1 n^1 + a_0$ então $T(n) = O(n^k)$.

Prova. Escolha $n_0 = 1$ e $c = |a_k| + \dots + |a_1| + |a_0|$.

Nós temos que mostrar que $\forall n \geq n_0, T(n) \leq c \cdot n^k$.

$$T(n) \leq |a_k|n^k + \dots + |a_1|n^1 + |a_0|n^0.$$

$$T(n) \leq |a_k|n^k + \dots + |a_1|n^k + |a_0|n^k.$$

$$T(n) \leq (|a_k| + \dots + |a_1| + |a_0|)n^k.$$

$$T(n) \leq c \cdot n^k \blacksquare$$

<https://www.geogebra.org/classic/dhycydkx>

